

CSE 4128

Image Processing and Computer Vision Laboratory

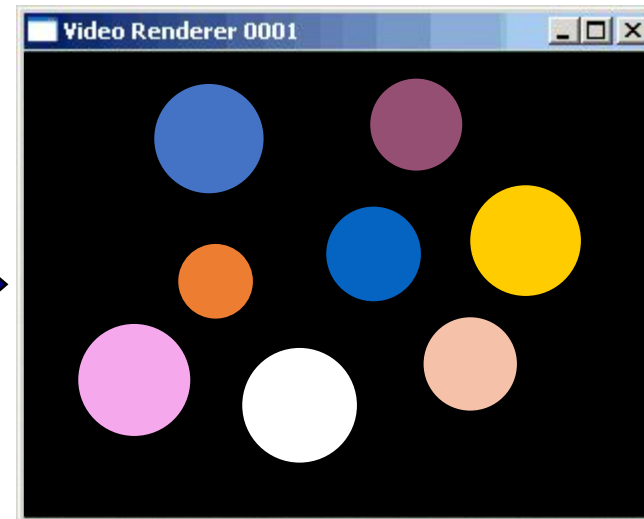
Week 2: Segmentation(Edge Detection)

Dr. Sk. Md. Masudul Ahsan
Professor
Dept. of CSE, KUET

Md Tajmilur Rahman
Lecturer
Dept. of CSE, KUET

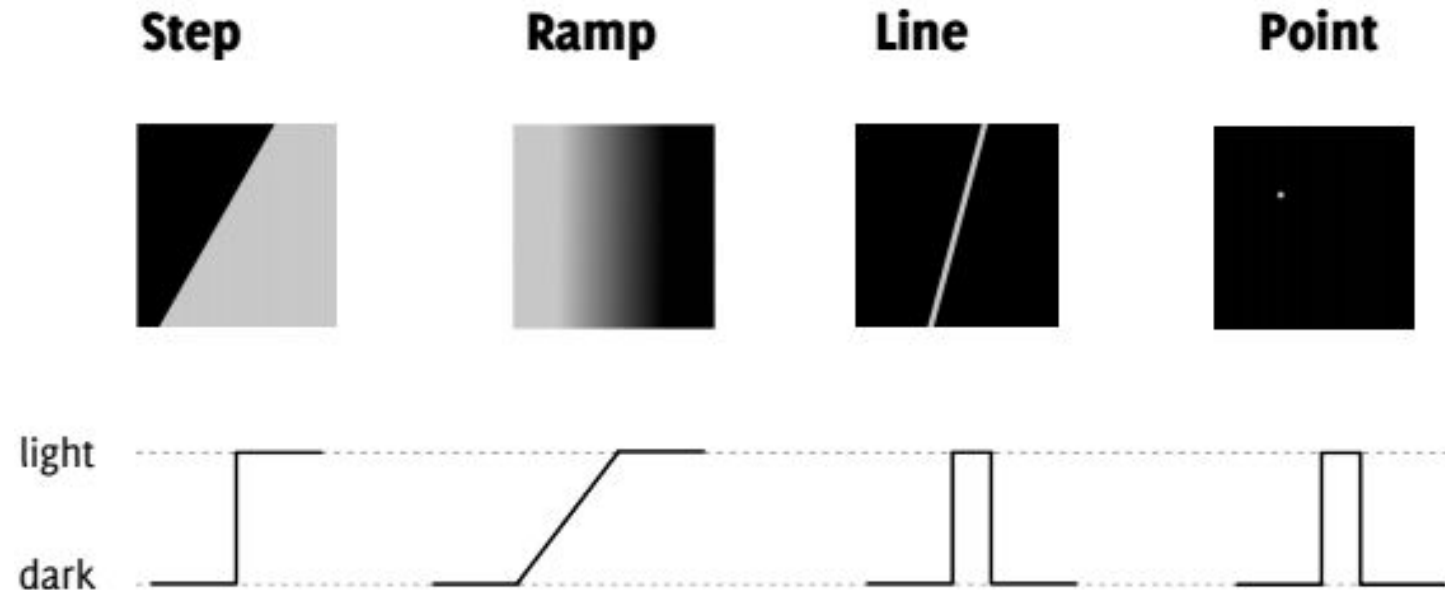
Segmentation

- Segmentation subdivides an image into its constituent regions or objects.
- Separating objects from the background and giving them individual labels(ID numbers)
- Segmentation attempts to partition the pixels of an image into groups that strongly correlate with the objects in an image



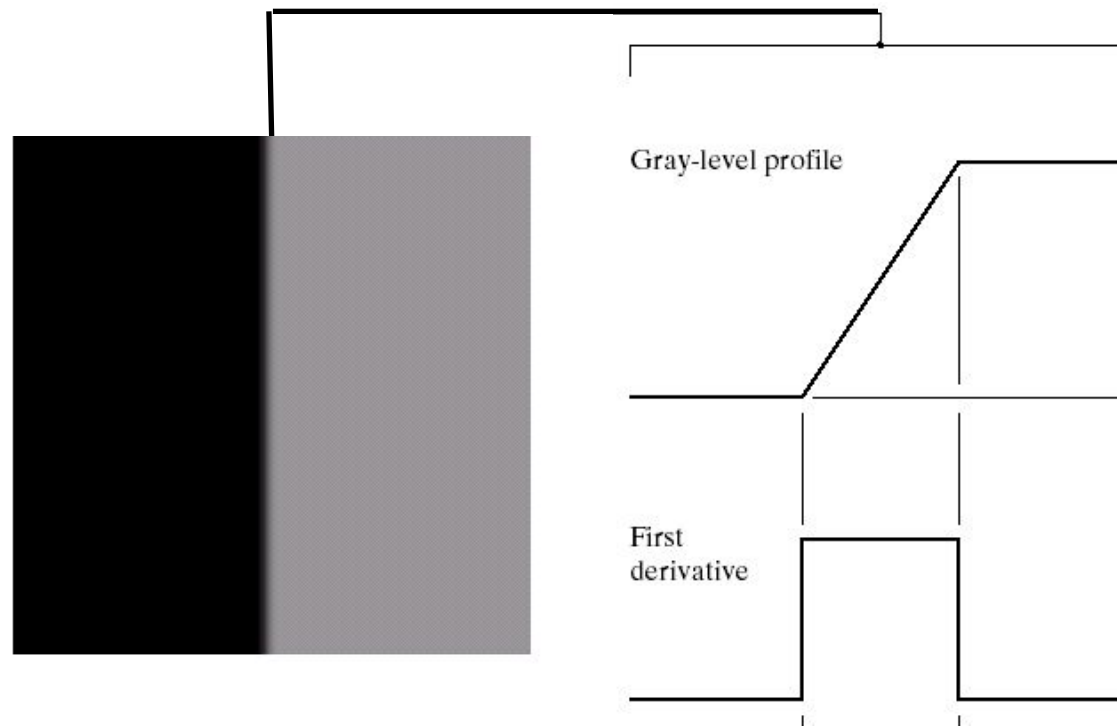
Edge Detection

- It is most frequently used for segmenting images based on abrupt(local) changes in intensity. It centers around contour detection.
- An edge is a set of connected pixels that lie on the boundary between two regions
- Edge models



Edge and Derivatives

- Derivatives are used to find discontinuities.
- 1st derivative tells us where an edge is.



Segmentation with Edge Detection

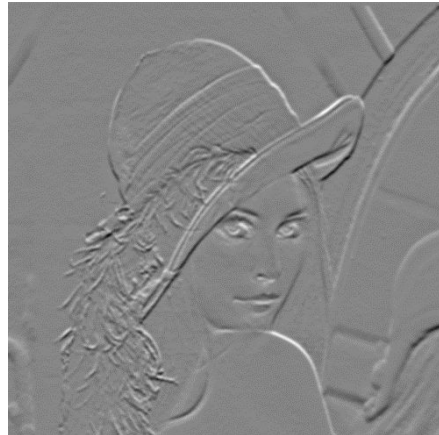


$$\frac{\partial f}{\partial x}$$

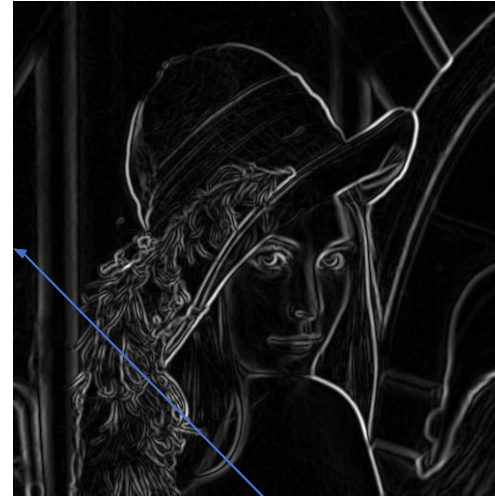


X-Derivative

$$\frac{\partial f}{\partial y}$$



Y-Derivative



Gradient Magnitude

$$\sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Applying
thresholding



Edge Detected

Gradient

- A 2-dimensional continuous Gaussian filter is defined as follows:

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

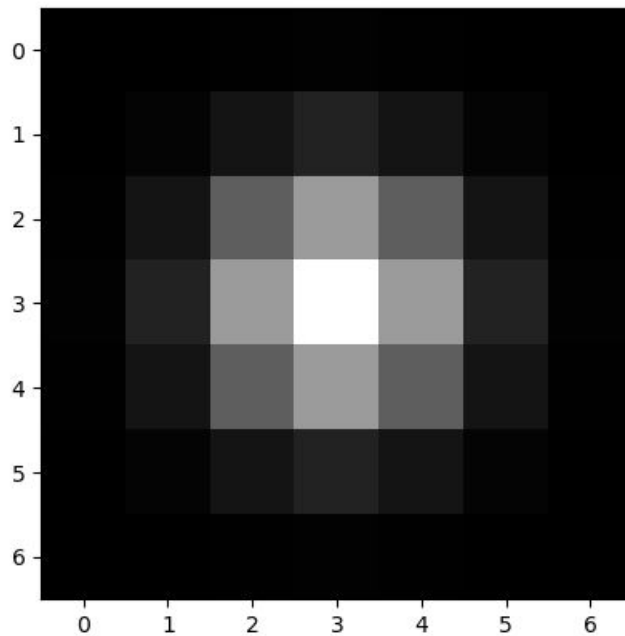
- For the 2-dimensional Gaussian $G_{\sigma}(x, y)$ the 1st order partial derivatives are:

$$\frac{\partial G_{\sigma}(x, y)}{\partial x} = -\frac{x}{\sigma^2} G_{\sigma}(x, y) \qquad \frac{\partial G_{\sigma}(x, y)}{\partial y} = -\frac{y}{\sigma^2} G_{\sigma}(x, y)$$

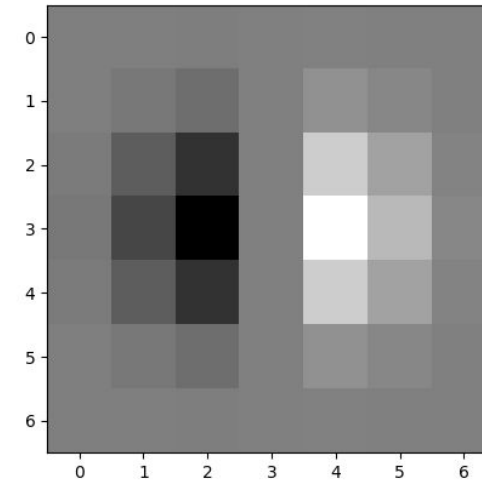
- The edge strength can be obtained from gradient magnitude:

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

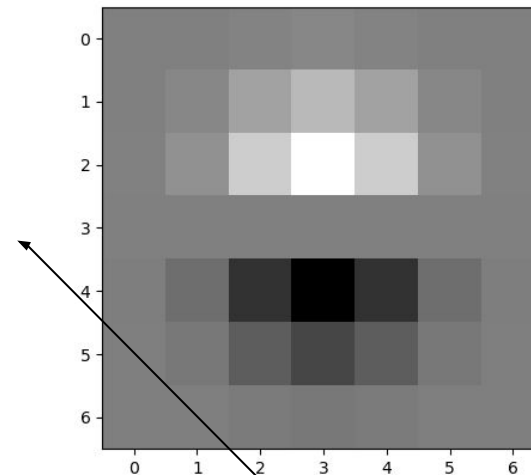
Gaussian filter & its Derivatives



7X7 kernel with sigma = 1



x-direction



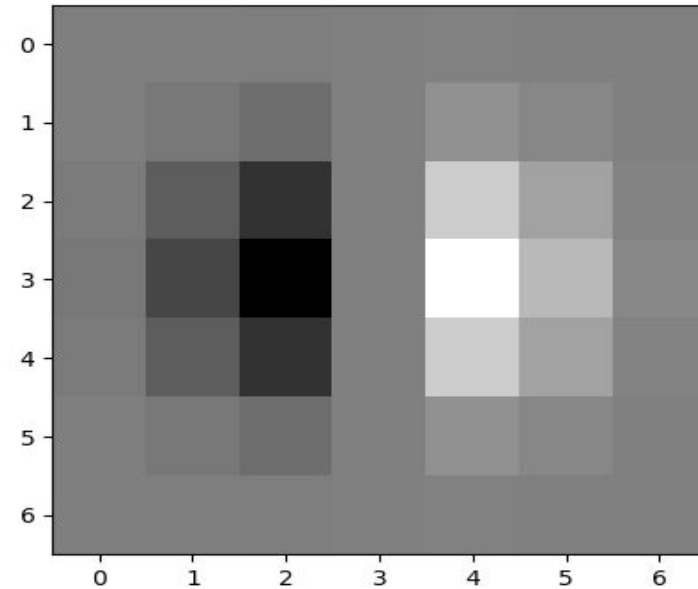
y-direction

X-derivative kernel

$$\frac{\partial G_{\sigma}(x, y)}{\partial x} = -\frac{x}{\sigma^2} G_{\sigma}(x, y)$$

0	0	0	0	0	0	0
0	-2	-3	0	3	2	0
-1	-7	-15	0	15	7	1
-1	-11	-25	0	25	11	1
-1	-7	-15	0	15	7	1
0	-2	-3	0	3	2	0
0	0	0	0	0	0	0

Normalized values (-25,25)



X-derivative kernel

-2.8e+12	-2.3e+13	-5.1e+13	-0	5.1e+13	2.3e+13	2.8e+12
-3.4e+13	-2.8e+14	-6.2e+14	-0	6.2e+14	2.8e+14	3.4e+13
-1.5e+14	-1.2e+15	-2.8e+15	-0	2.8e+15	1.2e+15	1.5e+14
-2.5e+14	-2e+15	-4.6e+15	-0	4.6e+15	2e+15	2.5e+14
-1.5e+14	-1.2e+15	-2.8e+15	-0	2.8e+15	1.2e+15	1.5e+14
-3.4e+13	-2.8e+14	-6.2e+14	-0	6.2e+14	2.8e+14	3.4e+13
-2.8e+12	-2.3e+13	-5.1e+13	-0	5.1e+13	2.3e+13	2.8e+12

Actual kernel values for sigma = 1

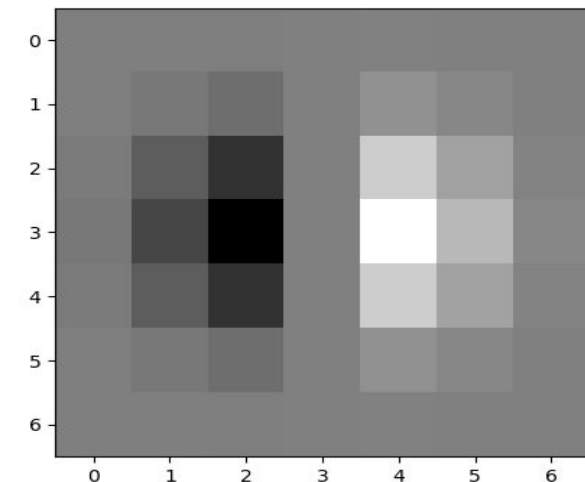
$$\frac{\partial G_{\sigma}(x, y)}{\partial x} = -\frac{x}{\sigma^2} G_{\sigma}(x, y)$$

```

0  0  0  0  0  0  0
0 -2 -3  0  3  2  0
-1 -7 -15 0 15 7 1
-1 -11 -25 0 25 11 1
-1 -7 -15 0 15 7 1
0 -2 -3  0  3  2  0
0  0  0  0  0  0  0

```

Normalized values (-25,25)

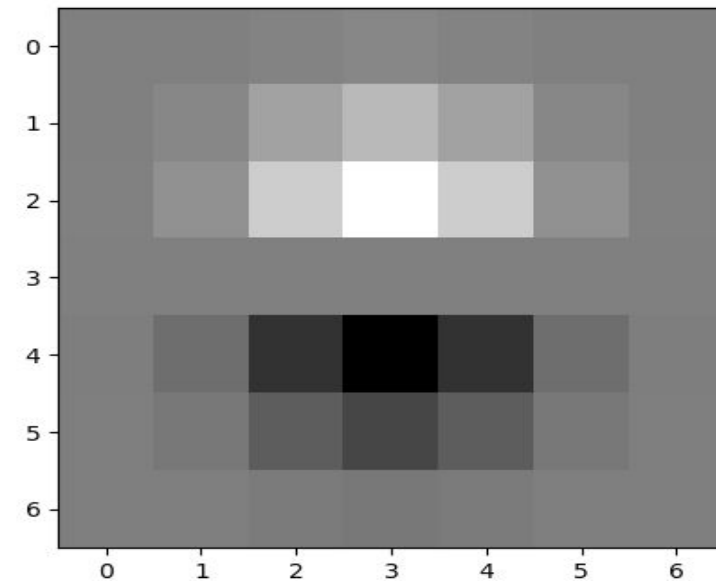


Y-derivative kernel

$$\frac{\partial G_{\sigma}(x, y)}{\partial y} = -\frac{y}{\sigma^2} G_{\sigma}(x, y)$$

0	0	1	1	1	0	0
0	2	7	11	7	2	0
0	3	15	25	15	3	0
0	0	0	0	0	0	0
0	-3	-15	-25	-15	-3	0
0	-2	-7	-11	-7	-2	0
0	0	-1	-1	-1	0	0

Normalized values (-25,25)



Y-derivative kernel

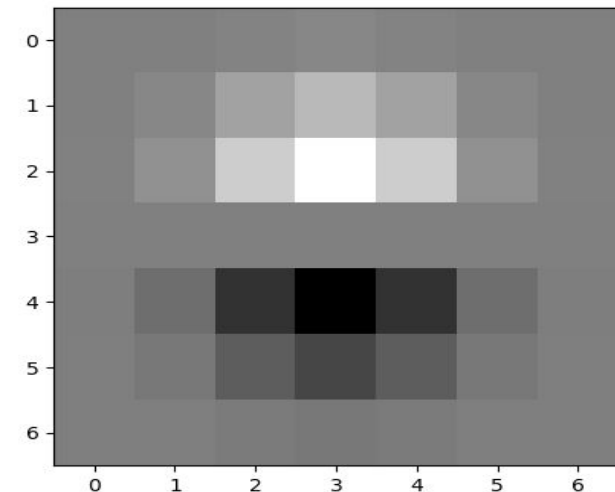
1.7e+12	2.1e+13	9.3e+13	1.5e+14	9.3e+13	2.1e+13	1.7e+12
1.4e+13	1.7e+14	7.6e+14	1.3e+15	7.6e+14	1.7e+14	1.4e+13
3.1e+13	3.8e+14	1.7e+15	2.8e+15	1.7e+15	3.8e+14	3.1e+13
0	0	0	0	0	0	0
-3.1e+13	-3.8e+14	-1.7e+15	-2.8e+15	-1.7e+15	-3.8e+14	-3.1e+13
-1.4e+13	-1.7e+14	-7.6e+14	-1.3e+15	-7.6e+14	-1.7e+14	-1.4e+13
-1.7e+12	-2.1e+13	-9.3e+13	-1.5e+14	-9.3e+13	-2.1e+13	-1.7e+12

Actual kernel values for sigma = 1

$$\frac{\partial G_{\sigma}(x, y)}{\partial y} = -\frac{y}{\sigma^2} G_{\sigma}(x, y)$$

0	0	1	1	1	0	0
0	2	7	11	7	2	0
0	3	15	25	15	3	0
0	0	0	0	0	0	0
0	-3	-15	-25	-15	-3	0
0	-2	-7	-11	-7	-2	0
0	0	-1	-1	-1	0	0

Normalized values (-25,25)

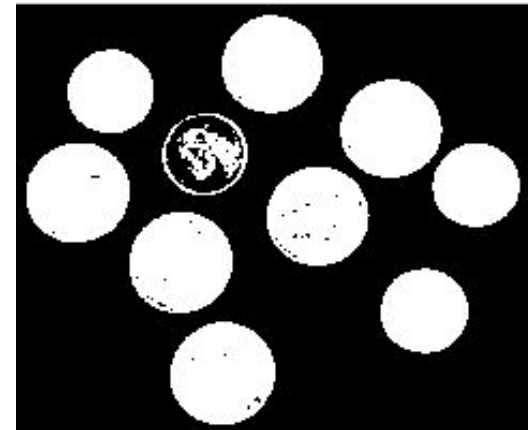
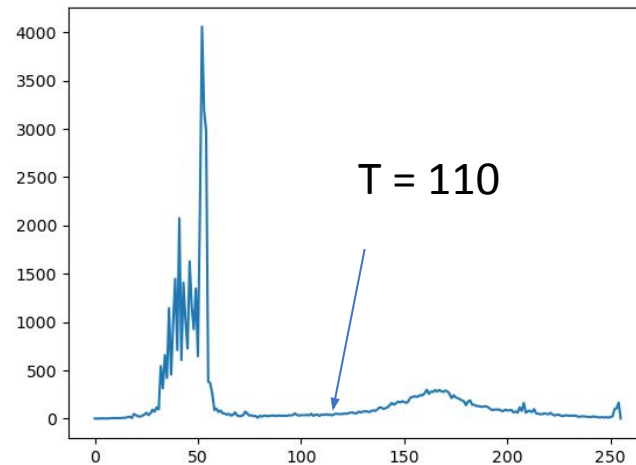


Thresholding

- By selecting an adequate threshold value T , the gray level image can be converted to binary image.

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases}$$

- Global Thresholding partitions the image histogram using a **single global threshold T** midway between the two gray value distributions



Double Thresholding

- We use **two** thresholds:
 - High threshold (T_{high})
 - Low threshold (T_{low})
- Given a gradient magnitude image:

1. Strong edges

1. Pixels with gradient magnitude $\geq T_{\text{high}}$
2. Immediately marked as **strong edges** (255).

2. Weak edges

1. Pixels with gradient magnitude between T_{low} and T_{high}
2. Marked as **weak edges** (often 128).

3. Non-edges

1. Pixels with gradient magnitude $< T_{\text{low}}$
2. Suppressed to **0**.

Workflow

1. Generate kernels for 1st order derivatives (x and y) for Gaussian filter, the kernel generation function **takes sigma as parameter** and generates kernels of size **7xsigma**.
2. Apply convolution on the image using the x and y derivative kernels.
3. Calculate the gradient magnitude from gradient images gx and gy:

```
gradient_magnitude = cv2.magnitude(gx.astype(np.float32),  
                                     gy.astype(np.float32))
```
4. Apply *double thresholding* on the gradient magnitude.
5. The resultant image shows the edges of the input image.

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad \frac{\partial G_{\sigma}(x, y)}{\partial x} = -\frac{x}{\sigma^2} G_{\sigma}(x, y) \quad \frac{\partial G_{\sigma}(x, y)}{\partial y} = -\frac{y}{\sigma^2} G_{\sigma}(x, y)$$

Output Results with different sigma value



sigma = 1 (kernel = 7X7)



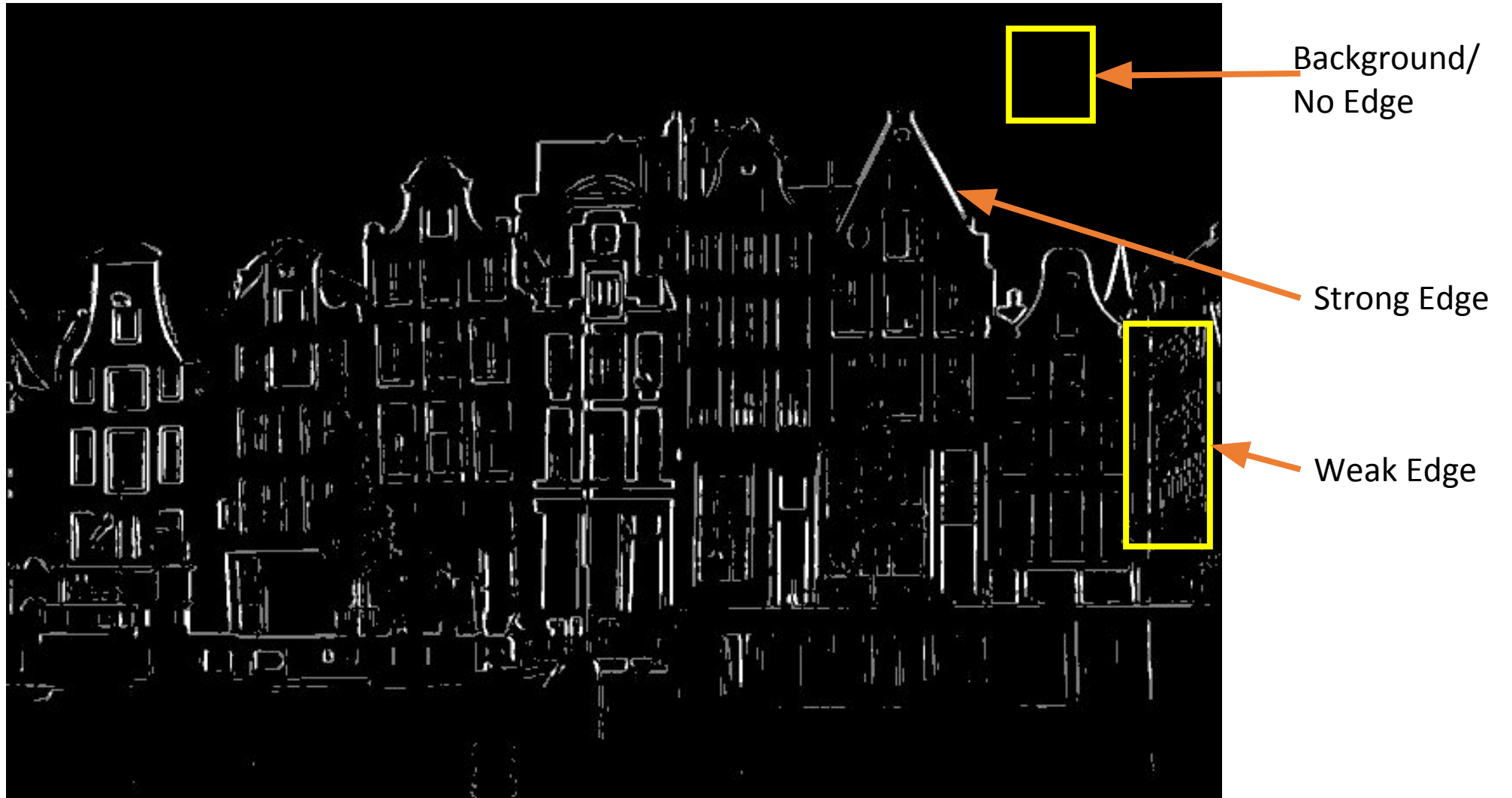
sigma = 2 (kernel = 13X13)

T_low = 100 T_high = 150

Another Input Image



Sigma = 1, T_l = 100, T_h = 150

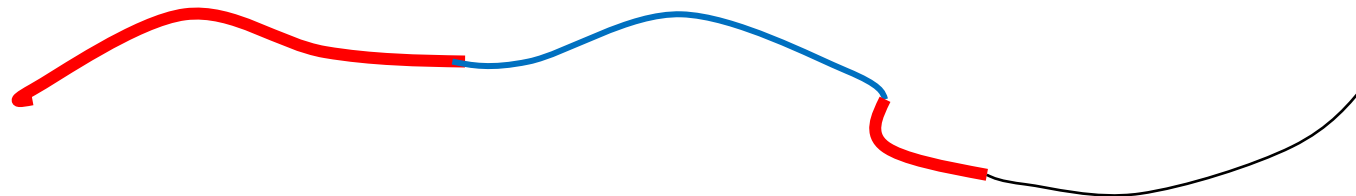


Assignment

1. Apply **Hysteresis Thresholding** instead of double thresholding for detecting edges in gradient magnitude image. Observe the differences with your class work and discuss it in your report.
2. Apply **Canny edge detection** algorithm on the image using the *opencv* library function. Compare the output with the previous ones.

Hysteresis Threshold

- Threshold at low/high levels to get weak/strong edge pixels
- Do connected components, starting from strong edge pixels
- Canny uses hysteresis threshold to fill up the narrow gap along the edge.
- Uses two threshold - high threshold T_h and low threshold T_l .
- Pixels with gray values greater than T_h is set as the initial edge pixels
- Pixels are set as background if their values are lower than T_l .



**Thank
You**